COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE010 - Data Structures and Algorithms	**SECTION:**	CPE21S1
DATE PERFORMED:	Oct 20, 2026	**DATE SUBMITTED:**	Oct 20, 2026
NAME:	Ignacio, Guillian Carlo V. Regala, Joaquin Luiz B.	**INSTRUCTOR:**	Engr. Jimlord Quejado

ACTIVITY NO. 14.1 [Algorithm Complexity]

1. PROCEDURE OUTPUT

	Best case	Worst case	Analysis (Ignacio)	Analysis(Regala)
Merge Sort Algorithm	O(n log n)	O(n log n)	Even if the algorithm is already sorted it would still use the recursion and divide and conquer at every level, which gives us O(n) and O(log n) when combined together we get O(n log n). So why is the worst case also O(n log n) because there is no bad input and merge sort will still do its job with no interruptions because it splits the elements into sub arrays which can be used for the conquering or combining at the end.	Even if the list is already sorted, the algorithm follows the exact same process: it recursively splits the array into single-element pieces and then combines them back together. Because this division and merging process requires the same number of operations every time, its performance never changes based on the initial order of the data.
Quick Sort Algorithm	O(n log n)	O(n^2)	We have the best case as O(n log n) because the pivot lands in the middle every time giving us	Quicksort takes $O(n \log n)$ time in the best case when the pivot splits the list into equal

			the left and right partitions, but unlike merge sort this one has O(n^2) for its worst case because the pivot is the smallest or largest element so one partition has n-1 and the other one has 0.	halves. However, it degrades to O(n^2) in the worst case if the pivot repeatedly picks the smallest or largest element, creating highly unbalanced partitions.

Table 14-1. Best- and Worst-Case Analysis using Theoretical Tools

	Merge Sort Algorithm	Quick Sort Algorithm
Best case	Time: 1.9300 secs Memory: 123.196 Mb	Time: 2.1600 secs Memory: 122.716 Mb
Worst case	Time: 3.4700 secs Memory: 122.812 Mb	Time: 2.5600 secs Memory: 123.404 Mb

Table 14-2. Best- and Worst-Case Analysis using Experimental Tools

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Theoretical Analysis		
	Binary Search	**Linear Search**
Pseudocode	BinarySearchArray(arr[], n, target) low ← 0 up ← n - 1 WHILE low ≤ up DO mid ← (low + up) / 2 // integer division IF arr[mid] = target THEN PRINT "Element is found" RETURN ELSE IF target < arr[mid] THEN up ← mid - 1 ELSE low ← mid + 1	LinearSearchArray(data[], n, target) i ← 0 WHILE i < n DO IF data[i] = target THEN PRINT "Searching is successful" RETURN END IF i ← i + 1 END WHILE PRINT "Searching is unsuccessful" END

	END IF END WHILE PRINT "Search element is not found" END BinarySearchLinkedList(start, target) low ← start high ← NULL found ← FALSE WHILE low ≠ high DO // find the middle node using slow/fast pointers slow ← low fast ← low WHILE fast ≠ high AND fast.next ≠ high DO slow ← slow.next // 1 step fast ← fast.next.next // 2 steps END WHILE IF slow.data = target THEN PRINT "Search element is found" found ← TRUE BREAK ELSE IF slow.data < target THEN low ← slow.next ELSE high ← slow END IF END WHILE IF found = FALSE THEN PRINT "Search element is not found" END IF END	LinearSearchLinkedList(head, target) current ← head WHILE current ≠ NULL DO IF current.data = target THEN PRINT "Searching is successful" RETURN END IF current ← current.next END WHILE PRINT "Searching is unsuccessful" END
Time complexity / Space complexity	**Time complexity:** **Best: O(1)** **Worst: O(log n)** **Space complexity:** **Iterative: O(1)**	**Time complexity:** **Best: O(1)** **Worst: O(n)** **Space complexity:** **Iterative: O(1)**
Analysis (Ignacio)	As we can see the best time complexity is O(1) because once we search the array that is sorted we could easily find it while it still has an O(log n) worst time complexity because it has to cut the array down in half	We have the best case as O(1) if the element we are trying to find is at the very first level while it has an O(n) worst case because it still has to search linearly or sequentially for the element inside the

	to search for a certain element.	unsorted or sorted array.
Analysis (Regala)	This has the best case time complexity of O(1) because the target can be found in the first comparison. Its worst case time complexity is O(n log n) because it repeatedly divides the search area in half.	This codes best case time complexity of O(1) because the target can be found in the first comparison. And its worst case time complexity is O(n) because it needs to check every element in the list.

Experimental Analysis	
Input size (N)	**Elapsed time for Binary Search**
1000	1000 Binary search (array) 0.157 0.077 1000 Binary search (linked list) 6.388 6.382
10000	10000 Binary search (array) 0.184 0.138 10000 Binary search (linked list) 76.066 67.738
100000	100000 Binary search (array) 0.199 0.170 100000 Binary search (linked list) 879.716 877.278

Input size (N)	**Elapsed time for Linear Search**
1000	1000 Linear search (array) 2.973 2.273 1000 Linear search (linked list) 5.139 3.822
10000	10000 Linear search (array) 29.268 3.694 10000 Linear search (linked list) 59.160 6.293
100000	100000 Linear search (array) 287.135 107.171 100000 Linear search (linked list) 646.527 214.490

SUPPLEMENTARY ANALYSIS	
Ignacio	Binary Search works better with little elements inside the array or list because the time increases

	drastically as the input size rises 10x while linear search has a better performance during large arrays or lists but is a bit slow for handling small datasets. Since binary search needs to have sorted elements for it to work better and for it to achieve its best case the linear search works better in unsorted arrays so their uses depend on the dataset we are actually implementing it to.
Regala	Binary Search is faster than the Linear Search because it divides the search area in half every time. As the input increases the size, binary search needs fewer comparisons, while linear Search needs to check every element one by one. However, binary search requires sorted data, while linear search can work with unsorted data.

CONCLUSION	
Ignacio	I therefore conclude that the run time depends on the resources of the CPU being used. The more we have background apps, or tabs running the slower the runtime of our program. Like the difference between linear search and binary search. They both have their best cases and worst cases. Merge sort has consistent performance that's why its runtime is almost consistent while Quicksort depends on the pivot for its performance.
Regala	In this activity, we learned how to analyze the time complexity of algorithms and understand how their performance changes depending on the input size. We compared Merge Sort and Quick Sort by identifying their best-case and worst-case time complexities. We also learned that Merge Sort has consistent performance, while Quick Sort's performance depends on its pivot selection. Overall, this activity helped us understand the importance of choosing efficient algorithms when sorting large amounts of data. We can improve more by practicing algorithm analysis and measuring execution times accurately.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **None**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **N/A**

COMPUTER ENGINEERING

QUEZON CITY

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **N/A**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.